

Fundamentos del Diseño y Modelado de Datos

Jorge Antonio Vidal Orozco

Jorge Eduardo Diaz Leyton

Samuel Giovanni Colmenares Patiño

Jhonatan Winston Sotelo Santofimio

Universidad de la Sabana

Diseño Conceptual y Lógico de la Base de Datos del Proyecto

Miguel Alfonso Varela

Mayo 11, 2026

Resumen Ejecutivo

El equipo se desempeña completamente en el área de tecnología, en la cual se han identificado las siguientes capacidades dentro del mismo para asegurar el éxito del proyecto:

1. Visión Arquitectónica

Capacidad para definir patrones de diseño (como Clean Architecture) que faciliten la integración con las bases de datos.

2. Dominio Técnico

Conocimiento previo en el ecosistema de PostgreSQL para la integridad de datos y en soluciones NoSQL para el manejo de grandes volúmenes de información.

3. Gestión de Procesos

Habilidad para automatizar tareas y coordinar la comunicación asíncrona del equipo.

Por lo anteriormente descrito y para el desarrollo completo del diseño y puesta en operación de la base de datos de Ecommify la selección de una arquitectura de alta disponibilidad se fundamenta en los requisitos operativos de una plataforma multivendedor con:

- **Continuidad del Negocio (PostgreSQL):** Se implementará con replicación y particionamiento para los datos críticos (pedidos y pagos). Esto garantiza que, ante un fallo en el nodo principal, el sistema pueda seguir operando sin pérdida de información transaccional.
- **Escalabilidad Horizontal (MongoDB):** Para manejar el alto volumen de reseñas y el comportamiento de los usuarios en tiempo real, se utilizará MongoDB con replica sets y

sharding. Esto permite distribuir la carga analítica y asegurar respuestas rápidas a medida que el catálogo crece.

- **Gestión de Compromisos (Teorema CAP):** El diseño se centrará en analizar los trade-offs entre consistencia y disponibilidad. Priorizaremos la consistencia en el módulo transaccional y la disponibilidad en el módulo de descubrimiento de productos, adaptándonos a las necesidades de un entorno distribuido.

Abstract

El presente documento tiene como objetivo exponer la solución arquitectónica propuesta para integrar motores de bases de datos relacionales y no relacionales en la plataforma de comercio electrónico Ecommify. La solución combina PostgreSQL como base de datos relacional y MongoDB como base de datos no relacional, con el propósito de mejorar el rendimiento, la escalabilidad y la disponibilidad del sistema. Debido al crecimiento exponencial de la información en módulos de alta demanda, como el catálogo de productos y las reseñas de clientes, depender únicamente de un motor relacional puede generar sobrecarga operativa, incremento en la latencia y posibles problemas de disponibilidad en el futuro.

Para afrontar estos desafíos, la arquitectura propuesta establece a PostgreSQL como la fuente principal de verdad, encargada de garantizar la integridad y consistencia de los datos transaccionales críticos del negocio. Por otra parte, MongoDB se incorpora para almacenar y gestionar información de alto volumen y acceso frecuente mediante documentos JSON, permitiendo operaciones de lectura más rápidas y una mayor escalabilidad horizontal.

Adicionalmente, la arquitectura implementa el patrón Change Data Capture (CDC), el cual permite sincronizar ambos motores de bases de datos. A través de CDC, cada cambio detectado en PostgreSQL es procesado y replicado automáticamente en MongoDB, garantizando consistencia de datos, optimización del rendimiento y soporte al crecimiento futuro de la plataforma.

Análisis de Requisitos

Ecommify es una plataforma de comercio electrónico de última generación orientada al mercado minorista global, diseñada para soportar millones de usuarios activos diarios y un catálogo dinámico con millones de artículos. La plataforma opera bajo un modelo omnicanal que permite la actualización en tiempo real. Para sostener el crecimiento exponencial del negocio y picos de tráfico críticos, como campañas de Black Friday, Ecommify requiere migrar su infraestructura hacia una arquitectura de persistencia políglota y de alta disponibilidad, eliminando puntos únicos de falla.

El núcleo tecnológico de Ecommify se dividirá estratégicamente entre PostgreSQL y MongoDB para aprovechar las fortalezas óptimas de cada motor. PostgreSQL actúa como el sistema de registro transaccional y financiero (sistema de control), garantizando el cumplimiento estricto de las propiedades ACID para la gestión de órdenes de compra, procesamiento de facturas, balances de cuentas e inventario centralizado. Por su parte, MongoDB se establece como la base de datos operativa y de lectura intensiva (sistema de engagement), encargándose del manejo del alto volumen de reseñas, catálogo de productos y la persistencia de carritos de compra. Ambas bases de datos se configuran en topologías distribuidas con replicación automática para asegurar resiliencia y disponibilidad continua del servicio.

Requisitos Funcionales (RF)

- RF1: El sistema debe gestionar un catálogo de productos multiclase con atributos dinámicos y variantes (talla, color, especificaciones técnicas) almacenados de forma flexible.

- RF2: La plataforma debe permitir a los usuarios autenticados agregar, modificar y persistir artículos en un carrito de compras con persistencia instantánea.
- RF3: El sistema debe procesar el flujo de checkout asegurando la inmutabilidad de la orden de compra una vez finalizada la transacción financiera.
- RF4: Se debe actualizar y sincronizar el inventario físico disponible de manera estrictamente transaccional en el momento exacto de confirmarse el pago.
- RF5: La base de datos debe almacenar perfiles de usuario unificados conteniendo direcciones de envío, métodos de pago preferidos.
- RF6: El sistema debe generar reportes financieros consolidados y auditorías de transacciones sin discrepancias numéricas para el módulo de contabilidad.
- RF7: El sistema debe guardar el historial completo de estados de envío de cada pedido con actualizaciones de rastreo provistas por proveedores logísticos.

Requisitos No Funcionales (RNF)

- RNF1: La arquitectura de base de datos debe garantizar una alta disponibilidad del 99.99% (SLA) mediante clústeres replicados y failover automático en ambas tecnologías.
- RNF2: El tiempo de respuesta para las consultas de lectura del catálogo de productos debe ser inferior a 50 milisegundos, utilizando estrategias de indexación avanzada.
- RNF3: El almacenamiento transaccional en PostgreSQL debe cumplir con el estándar estricto de aislamiento ACID para prevenir colisiones de concurrencia en compras simultáneas.
- RNF4: La infraestructura de datos debe contar con mecanismos de particionamiento y sharding horizontal para escalar la capacidad de almacenamiento ante incrementos masivos del catálogo.

- RNF5: Todos los datos en tránsito y en reposo (especialmente información de pago) deben estar encriptados mediante algoritmos AES-256 y TLS 1.3.

Identificación de Entidades del Dominio

- geolocations
- order_statuses
- payment_types
- categories
- customers
- sellers
- products
- orders
- order_items
- order_payments
- reviews

Tabla de volúmenes de datos esperados

Con una proyección de crecimiento escalado en 1000 usuarios por día.

- Usuarios/Clientes: 30,000 nuevos registros/mes.
- Vendedores: 1,500 nuevos vendedores/mes.
- Productos: 6,000 nuevos productos/mes (con 2 fotos promedio = 12,000 fotos/mes).
- Pedidos (Orders): 1.5 pedidos promedio por usuario al mes = 45,000 pedidos/mes.
- Ítems por pedido: 2 productos por orden = 90,000 registros/mes.
- Pagos: 1.1 pagos por pedido = 49,500 registros/mes.
- Reseñas: 30% de los pedidos reciben reseña = 13,500 registros/mes.

Nombre de la Tabla	Peso Registro (Bytes)	Crecimiento (Registros/Mes)	Espacio Mensual	Registros a 1 Año	Espacio Total 1 Año (Con Índices)
geolocations	76 B	0 (<i>Estática</i>)	0 MB	50,000	5.32 MB (<i>Inicial</i>)
order_statuses	54 B	0 (<i>Estática</i>)	0 MB	8	0.6 KB (<i>Inicial</i>)
payment_types	54 B	0 (<i>Estática</i>)	0 MB	5	0.3 KB (<i>Inicial</i>)
categories	104 B	10 / mes	0.001 MB	120	17.4 KB
customers	170 B	30,000 / mes	5.10 MB	360,000	85.68 MB
sellers	78 B	1,500 / mes	0.12 MB	18,000	1.96 MB
products	148 B	6,000 / mes	0.89 MB	72,000	14.92 MB
orders	48 B	45,000 / mes	2.16 MB	540,000	36.29 MB
order_items	40 B	90,000 / mes	3.60 MB	1,080,000	60.48 MB
order_payments	28 B	49,500 / mes	1.39 MB	594,000	23.28 MB

reviews	248 B	13,500 / mes	3.35 MB	162,000	56.25 MB
---------	-------	--------------	---------	---------	----------

Diseño conceptual

Tabla: geolocations

- Atributos:
 - zip_code_prefix (INT): Identificador único (PK).
 - location (GEOMETRY(Point, 4326)): latitud/longitud
 - city (VARCHAR 50): Nombre de la ciudad (NN).
 - state (VARCHAR 2): Siglas del estado (NN).
- Cardinalidad: 1:N con customers y sellers.

Tabla: order_statuses

- Atributos:
 - id (SERIAL): Identificador autoincremental (PK).
 - name (VARCHAR 50): Nombre del estado (NN, UNIQUE).
- Cardinalidad: 1:N con orders.

Tabla: payment_types

- Atributos:
 - id (SERIAL): Identificador autoincremental (PK).
 - name (VARCHAR 50): Tipo de pago (NN, UNIQUE).
- Cardinalidad: 1:N con order_payments.

Tabla: categories

- Atributos:
 - id (SERIAL): Identificador autoincremental (PK).
 - name (VARCHAR 100): Nombre de categoría (NN, UNIQUE).

- Cardinalidad: 1:N con products.

Tabla: customers

- Atributos:
 - identification_number (VARCHAR 50): ID del cliente (PK).
 - geolocation_zip_code_prefix (INT): Referencia postal (FK).
 - name (VARCHAR 50): Nombre (NN).
 - last_name (VARCHAR 50): Apellido (NN).
 - phone_number (VARCHAR 10): Teléfono (NN).
 - address (ADDRESS): Dirección física (NN).
- Relación: Depende de geolocations.

Tabla: sellers

- Atributos:
 - id (SERIAL): ID del vendedor (PK).
 - zip_code_prefix (INT): Referencia postal (FK).
 - name (VARCHAR 50): Nombre (NN).
 - last_name (VARCHAR 50): Apellido (NN).
 - phone_number (VARCHAR 10): Teléfono (NN).
- Relación: Depende de geolocations.

Tabla: products

- Atributos:
 - id (SERIAL): ID del producto (PK).
 - category_id (INT): Referencia categoría (FK).
 - name (VARCHAR 100): Nombre producto (NN).
 - specifications (JSONB): Especificaciones del producto.
 - photos (TEXT[]): Ruta del sistema donde se guarda cada foto del producto.

- Relación: Depende de categories.

Tabla: orders

- Atributos:
 - id (SERIAL): ID de pedido (PK).
 - customer_identification_number (VARCHAR 50): Cliente (FK).
 - order_status_id (INT): Estado actual (FK).
 - purchase_timestamp (TIMESTAMP): Fecha compra (NN).
 - approved_at, delivered_carrier_date, delivered_customer_date, estimated_delivery_date (TIMESTAMP).
- Relación: Conecta clientes con estados de pedido.

Tabla: order_items

- Atributos:
 - id (SERIAL): ID línea de pedido (PK).
 - product_id (INT): Producto (FK).
 - seller_id (INT): Vendedor (FK).
 - order_id (INT): Pedido (FK).
 - quantity (INT): Cantidad (NN, CHECK > 0).
 - price, freight_value (FLOAT): Costos (NN, CHECK >= 0).
 - created_at (TIMESTAMP)
- Relación: Tabla de unión N:M (implícita) entre orders, products y sellers.

Tabla: order_payments

- Atributos:
 - id (SERIAL): ID línea de pedido (PK).
 - order_id (INT): Pedido (FK).
 - payment_type_id (INT): Tipo de pago (FK).

- sequential (INT): Número de pago (PK parte 1).
- installments (INT): Cuotas (CHECK > 0).
- value (FLOAT): Monto (CHECK > 0).
- Relación: PK Compuesta (order_id + sequential).

Diseño Lógico

Tabla: geolocations

Atributo	Tipo de Dato	Restricción / Llave	Descripción
zip_code_prefix	INT	PK	Identificador único
location	GEOMETRY(Point, 4326)	NN	Latitud y longitud
city	VARCHAR(50)	NN	Nombre de la ciudad
state	VARCHAR(2)	NN	Siglas del estado

Tabla: order_statuses

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	PK	Identificador autoincremental

name	VARCHAR(50)	NN, UNIQUE	Nombre del estado
------	-------------	------------	-------------------

Tabla: payment_types

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	PK	Identificador autoincremental
name	VARCHAR(50)	NN, UNIQUE	Tipo de pago

Tabla: categories

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	PK	Identificador autoincremental
name	VARCHAR(100)	NN, UNIQUE	Nombre de categoría

Tabla: customers

Atributo	Tipo de Dato	Restricción / Llave	Descripción
identification_number	VARCHAR(50)	PK	ID del cliente

geolocation_zip_code_prefix	INT	FK (geolocations)	Referencia postal
name	VARCHAR(50)	NN	Nombre
last_name	VARCHAR(50)	NN	Apellido
phone_number	VARCHAR(10)	NN	Teléfono
address	ADDRESS	NN	Dirección física

Tabla: sellers

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	PK	ID del vendedor
zip_code_prefix	INT	FK (geolocations)	Referencia postal
name	VARCHAR(50)	NN	Nombre
last_name	VARCHAR(50)	NN	Apellido
phone_number	VARCHAR(10)	NN	Teléfono

Tabla: products

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	PK	ID del producto
category_id	INT	FK (categories)	Referencia categoría
name	VARCHAR(100)	NN	Nombre producto
specifications	JSONB	NN	Especificaciones del producto
photos	TEXT[]		Lista de fotos del producto

Tabla: orders

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	PK	ID de pedido
customer_identification_number	VARCHAR(50)	FK (customers)	Cliente

order_status_id	INT	FK (order_statuses)	Estado actual
purchase_timestamp	TIMESTAMP	NN	Fecha compra
approved_at	TIMESTAMP	—	Fecha aprobación
delivered_carrier_date	TIMESTAMP	—	Fecha transportadora
delivered_customer_date	TIMESTAMP	—	Fecha entrega cliente
estimated_delivery_date	TIMESTAMP	—	Fecha estimada
created_at	TIMESTAMP	NN DEFAULT CURRENT_TIME STAMP	

Tabla: order_items

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	PK	ID línea de pedido

product_id	INT	FK (products)	Producto
seller_id	INT	FK (sellers)	Vendedor
order_id	INT	FK (orders)	Pedido
quantity	INT	NN, CHECK > 0	Cantidad
price	FLOAT	NN, CHECK >= 0	Costo producto
freight_value	FLOAT	NN, CHECK >= 0	Costo flete
created_at	TIMESTAMP	NN DEFAULT CURRENT_TIMESTAMP	

Tabla: order_payments

Atributo	Tipo de Dato	Restricción / Llave	Descripción
id	SERIAL	—	ID línea de pago
order_id	INT	FK (orders), PK Parte 1	Pedido
payment_type_id	INT	FK (payment_types)	Tipo de pago

sequential	INT	PK Parte 2	Número de pago correlativo
installments	INT	CHECK > 0	Cuotas
value	FLOAT	CHECK > 0	Monto

Extensiones a Utilizar

Campo	Extensión	Justificación
products.name	pg_trgm	- Los usuarios suelen cometer errores de tipado u ortografía y operadores como like o = fallan al mostrar resultados. - Con la extensión puedes tener resultados similares con el operador <->. - Permite buscar texto completo de forma casi instantánea y puede utilizar índices GIN y GiST.
geolocations.location	PostGIS	Guardando los datos en el formato de la extensión se puede calcular la distancia entre dos puntos y calcular el valor de los fletes con más

	precisión.
customers.addr	pgcrypto
ess	
customers.pho	
ne_number	
sellers.phone_	
number	

- Para cumplir con normas regulatorias y brindar seguridad a los clientes/vendedores de ecommify se guardan los datos encriptados.

Estrategia de particionamiento

La estrategia de particionamiento debe plantearse para las tablas de mayor crecimiento por fecha de creación de registro las cuales son, *orders* y *order_items*.

<i>Zona Hot</i>	<i>Zona Cold</i>
• Últimos 3 meses de operación. • Para lectura operativas y consultas. • Almacenamiento en SSD de alto rendimiento.	• Datos mayores a 3 meses. • Para consultas históricas. • El almacenamiento debería ser en discos mecánicos.

Diseño lógico preliminar - Módulo MongoDB

Colección MongoDB	Propósito	Justificación
products_catalog	Almacenar el catálogo de productos optimizado para búsquedas, filtros y consultas rápidas.	Permite manejar atributos dinámicos y estructuras flexibles utilizando documentos JSON desnormalizados.

reviews	Almacenar las reseñas y calificaciones realizadas por los clientes sobre los productos.	Alta frecuencia de lectura y escritura, ideal para MongoDB por su escalabilidad horizontal y acceso rápido.
recent_reviews_product	Guardar únicamente las reseñas más recientes de cada producto para mejorar el rendimiento de lectura.	Reduce el tamaño de los documentos y acelera la carga de información en la vista del producto.

Patrones de Modelado Aplicados

Para garantizar la eficiencia del sistema y el enfoque híbrido entre el SQL y NoSQL, se implementarán los siguientes patrones de diseño en NoSQL:

A. Patrón de Referencia Extendida

Dado que las reseñas se consultan frecuentemente junto con el nombre del producto y el nombre del cliente, evitar múltiples **"Join"** puede ser muy beneficioso para aliviar la carga de nuestra base de datos. Se incluyen datos básicos del producto y del cliente directamente en el documento de la reseña.

- **Estrategia:** Al momento de la creación de una reseña, la aplicación realiza un **JOIN único** en PostgreSQL para obtener el name del producto y el name del cliente. Estos datos se inyectan en el documento JSON que se persiste en la colección reviews de MongoDB.
- **Ventaja:** Se eliminan los saltos entre tablas (Customers -> Orders -> Order_Items -> Products) en las consultas de lectura. Esto permite obtener reseñas enriquecidas con

una sola operación de búsqueda en la base de datos documental, liberando de carga a la base de datos relacional.

Ejemplo de Colección reviews(JSON):

```
{
  "_id": "66479b...",
  "score": 5,
  "comment": "Excelente calidad, muy recomendado.",
  "created_at": "2026-05-17T18:00:00Z",
  "product_extended": {
    "product_id": "PROD-123",
    "name": "Camiseta Algodón Premium"
  },
  "customer_extended": {
    "customer_id": "CUST-456",
    "name": "Jhon Doe"
  }
}
```

B. Patrón de Subconjunto

Para evitar documentos excesivamente grandes en la colección de productos cuando un artículo tiene miles de reseñas, aplicamos el patrón de subconjunto.

- **Estrategia:** La colección principal de products en MongoDB almacenará únicamente las **10 reseñas más recientes**. El historial completo de reseñas residirá en la colección independiente reviews.

- **Ventaja:** Reduce el uso de memoria RAM en el servidor de base de datos y acelera la carga de la página de detalles del producto, que es la más visitada.

Ejemplo de Documento en Colección recent_reviews_product:

```
{
  "_id": "PROD-123",
  "name": "Camiseta Algodón Premium",
  "category": "Ropa",
  "overall_rating": 4.8,
  "total_reviews_count": 150,
  "recent_reviews": [
    {
      "review_id": "66479b...",
      "customer_name": "Jhon Doe",
      "score": 5,
      "comment": "Excelente calidad",
      "date": "2026-05-17"
    }
  ]
}
```

C. Patrón de diseño de captura de datos de cambios (CDC)

Con el objetivo de diseñar un catálogo de productos altamente eficiente y escalable, se implementará una estrategia de persistencia utilizando el patrón CDC.

Bajo este enfoque, PostgreSQL se mantendrá de forma estricta como nuestra fuente de la verdad, encargándose exclusivamente de las operaciones de escritura (creación, actualización y retiro de productos), garantizando la integridad transaccional del negocio. Por otro lado,

MongoDB asumirá el rol de vista de lectura, haciendo el 100% de la carga de consultas, búsquedas y filtros del catálogo.

- **Estrategia:** Para mantener la homologación y sincronización de datos entre ambos motores con la máxima eficiencia, el componente CDC monitorea en tiempo real el log de transacciones de PostgreSQL. Cada cambio detectado será transformado y enviado a MongoDB mediante eventos asíncronos.
- **Ventaja:** Al desviar el tráfico masivo de lecturas y búsquedas hacia MongoDB, la CPU y memoria de PostgreSQL se reservan para operaciones críticas como compras, pagos y gestión de inventario, eliminando cuellos de botella.

Ejemplo de Documento en Colección `products_catalog`:

```
{
  "_id": 12345,
  "name": "Nombre del Producto",
  "specifications": {
    "marca": "Sony",
    "color": "Negro",
    "almacenamiento": "1TB"
  },
  "photos": [
    "https://images.com/foto1.jpg",
    "https://images.com/foto2.jpg"
  ],
  "category": {
    "id": 1,
```

```
    "name": "Electrónicos"
  },
  "seller_summary": {
    "id": 9876,
    "name": "Nombre Vendedor",
    "last_name": "Apellido Vendedor",
    "phone_number": "1234567890",
    "zip_code_prefix": 760001
  }
}
```


¿Qué datos van a MongoDB vs PostgreSQL?

Para optimizar el rendimiento de la plataforma, se ha definido una arquitectura híbrida donde **PostgreSQL** actúa como la fuente principal de las transacciones y relaciones complejas, mientras que **MongoDB** se utiliza para gestionar datos de alta lectura y estructuras dinámicas, como el sistema de reseñas.

Motor	Datos	Justificación Técnica
PostgreSQL	Clientes, Geolocalización, Órdenes, Pagos, Vendedores, Productos, Estado de la orden.	Requiere estricta integridad referencial, cumplimiento de propiedades ACID y manejo de relaciones complejas para la lógica financiera y logística. Las actualizaciones de catálogo (precios, stock, altas) se consolidan aquí de forma segura.
MongoDB	Reseñas (Reviews), Catálogo de Productos Carrito de compra (Session)	Estos datos tienen una alta tasa de lectura. MongoDB permite almacenar documentos pre-agregados, eliminando la necesidad de realizar múltiples Joins en las tablas en tiempo de ejecución, que se traduce en el consumo de los recursos del hardware de la base de datos. Actúa como un "espejo de lectura" alimentado por eventos asíncronos vía CDC. Almacena la información del producto de forma desnormalizada (incluyendo categorías

y atributos dinámicos) para resolver búsquedas, filtros y visualizaciones de la aplicación en una sola operación de lectura de milisegundos.

Decisiones Arquitectónicas Justificadas

Matriz de Decisión por Entidad

Entidad	Motor de Base de Datos	Motivo de Selección
Address	PostgreSQL	Almacena datos estructurados de ubicación. Mantiene integridad referencial con clientes y vendedores.
Customers	PostgreSQL	Exige integridad referencial estricta. Previene registros huérfanos. Protege la identidad del usuario.
Geolocations	PostgreSQL	Soporta extensiones espaciales como PostGIS. Calcula distancias exactas para definir costos de envío.
Orders	PostgreSQL	Requiere cumplimiento ACID absoluto. Evita la pérdida de transacciones financieras.
Order Payments	PostgreSQL	Garantiza cumplimiento ACID estricto. Previene transacciones financieras duplicadas o pérdidas. Asegura la relación exacta con la orden de

		compra y el tipo de pago.
Products	PostgreSQL y MongoDB	PostgreSQL gestiona el inventario maestro con consistencia fuerte. MongoDB distribuye el catálogo para búsquedas rápidas de atributos variables.
Sellers	PostgreSQL	Requiere validación de relaciones con inventario y facturación. Garantiza consistencia en el ecosistema comercial.
Shopping Carts	MongoDB	Soporta alta velocidad de escritura y lectura. Persiste sesiones temporales sin bloquear el motor transaccional.
Product Reviews	MongoDB	Maneja esquemas flexibles para comentarios. Escala horizontalmente ante volúmenes masivos de datos sociales.
Recent Reviews	MongoDB	Aplica el patrón de subconjunto. Entrega las opiniones más recientes con latencia mínima.

Trade-offs

1. Complejidad Operacional frente a Rendimiento Especializado

Situación: Mantener dos motores de bases de datos mejora la velocidad de respuesta. Esta decisión incrementa drásticamente la carga de administración de la infraestructura y dificulta el rastreo de errores.

Estrategia de mitigación: Implementaremos Infraestructura como Código (IaC) para automatizar los despliegues. Centraliza el monitoreo de ambos motores en un panel de observabilidad único.

Motivo de la estrategia: La intervención manual genera tiempos de inactividad. Automatizar la infraestructura asegura despliegues repetibles y disminuye la carga operativa del equipo de ingeniería.

2. Consistencia Eventual frente a Disponibilidad Inmediata

Situación: Almacenar las reviews en un sistema distribuido asegura lecturas veloces. Los nuevos comentarios tardan milisegundos en replicarse en todos los nodos.

Estrategia de mitigación: Utilizaremos la sincronización asíncrona orientada a eventos. Muestra actualizaciones optimistas en la interfaz del usuario.

Motivo de la estrategia: El comercio electrónico tolera un retraso visual en los comentarios sociales. La plataforma no tolera una caída en el catálogo de productos. La sincronización en segundo plano protege el rendimiento del flujo principal de compra.

3. Costo de Infraestructura frente a Tolerancia a Fallos

Situación: Configurar replicación continua y tolerancia a fallos asegura una disponibilidad del 99.99%. Duplicar los nodos multiplica los costos de servidores y almacenamiento.

Estrategia de mitigación: Aplicaremos el particionamiento de tablas por rango de fechas para las órdenes. También moveremos la información histórica (anteriores a 6 meses) a discos de almacenamiento en frío, modalidad disponible en la mayoría de nubes públicas.

Aprovechando el movimiento de datos, mantendremos exclusivamente los datos transaccionales del mes actual en espacios de tabla de estado sólido; es decir, las tablas del mes en curso estarán en discos SSD y los meses históricos estarán en discos HDD.

Motivo de la estrategia: Los usuarios consultan información reciente casi en su totalidad. Mover los datos históricos reduce la factura mensual en la nube sin degradar la experiencia transaccional.

Teorema CAP

El Teorema CAP establece que en un sistema distribuido sólo es posible garantizar dos de las tres propiedades fundamentales: Consistencia (Consistency), Disponibilidad (Availability) y Tolerancia al Particionamiento (Partition Tolerance). En el contexto de Ecommify, al ser un sistema que opera sobre redes distribuidas, la tolerancia al Particionamiento es un requisito no negociable. Por lo tanto, la arquitectura debe elegir entre Consistencia y Disponibilidad para cada flujo de negocio específico.

Módulo de Pedidos y Pagos

La prioridad es la Consistencia (CP). Cada transacción financiera debe ser exacta y reflejar el estado real del inventario. El sistema prefiere rechazar una solicitud temporalmente antes que permitir un estado inconsistente en las finanzas del negocio.

Módulo de Catálogo y Búsqueda

La prioridad es la Disponibilidad (AP). Es crítico que los usuarios puedan navegar y descubrir productos en todo momento. Un retraso mínimo en la propagación de un cambio de precio es aceptable frente a una caída total del servicio de búsqueda.

Módulo de Reseñas y Calificaciones

La prioridad es la Disponibilidad (AP). Este módulo maneja un alto volumen de lecturas y escrituras sociales. La consistencia eventual es suficiente para asegurar que las opiniones de los usuarios se reflejan globalmente en cuestión de milisegundos sin bloquear la experiencia de navegación.

Módulo de Geolocalización

La prioridad es la Consistencia (CP). El cálculo de impuestos y costos de envío depende de datos precisos de ubicación. Un error en esta información impacta directamente en el margen operativo y el cumplimiento legal.

Estrategias de mitigación.

1. Consistencia Eventual vs Consistencia Fuerte

- a. **El riesgo:** Al actualizar un producto en PostgreSQL, ese cambio puede no reflejarse inmediatamente en MongoDB.
- b. **El beneficio:** Las lectura en el frontend de MongoDB son extremadamente rápidas porque el producto y sus reseñas pueden recuperarse en una sola consulta.

2. Duplicación de Datos e Integridad Referencial

- a. **El riesgo:** MongoDB no tiene llaves foráneas lo que implica que si un producto se elimina puede que sus reseñas queden huérfanas.

- b. **El beneficio:** Desacoplamiento total. El sistema puede funcionar con MongoDB aun si PostgreSQL entra en mantenimiento

3. Complejidad de Infraestructura y Desincronización

- a. **El riesgo:** Mantener dos motores requiere un patrón de sincronización.
- b. **El beneficio:** Escalabilidad independiente para MongoDB horizontalmente para soportar millones de reseñas y lecturas pesadas.

Para solventar los trade-offs indicados anteriormente y asegurar una arquitectura robusta de alta disponibilidad, se deben aplicar las siguientes estrategias:

- Implementación de CDC (Change Data Capture) utilizando herramientas como Debezium acoplado con Apache Kafka. Debezium lee el WAL (Write-Ahead Log) de PostgreSQL en tiempo real y publica los cambios de productos en un tópico de kafka. Un consumidor procesa el tópico y actualiza el documento.
- Guardar las reseñas en una colección independiente donde cada elemento tiene una referencia del `product_id` que viene de PostgreSQL.
- Prohibir borrados físicos con banderas. De esta forma el evento de desactivación viaja por el pipeline de Kafka y permitiendo ocultar el producto y sus reseñas sin romper la integridad referencial.
- Implementar un proceso de conciliación semanal en horas de bajo tráfico que compare mediante hashing y conteo de registros si hay discrepancias por pérdida de mensajes en Kafka. Si hay diferencias el script repara los documentos faltantes o desactualizados.

Flujos de sincronización si aplica.

Para mantener la coherencia entre ambos motores de base de datos sin comprometer la alta disponibilidad, Ecommify implementa el patrón Transactional Outbox. Este patrón garantiza que

los cambios en PostgreSQL (fuente de verdad) se propaguen de forma asíncrona hacia MongoDB.

Detalle del Funcionamiento

Cada vez que ocurre un evento crítico en PostgreSQL, como la creación/actualización/eliminación de un producto, el sistema escribe el cambio en una tabla técnica (Outbox) de mensajes dentro de la misma transacción de la base de datos. Un servicio de captura de datos de cambio (CDC) monitorea esta tabla y publica los eventos de forma asíncrona hacia MongoDB. Este proceso asegura que el sistema nunca pierda una actualización, incluso si hay fallos en la red o en alguno de los nodos.

Ventajas y Aplicación

Este patrón elimina el acoplamiento directo entre los servicios y mantiene la consistencia eventual. La base de datos transaccional no tiene que esperar a que la base de datos NoSQL confirme la recepción del dato para finalizar la compra, ya que los eventos permanecen seguros en la tabla Outbox. Shopify utiliza arquitecturas orientadas a eventos similares para sincronizar inventarios entre múltiples regiones y proveedores, garantizando que el stock siempre sea el correcto sin importar el volumen de tráfico.

Anexos

Diccionario de datos

1. Entidad: geolocations

- Descripción / Propósito: Almacena los puntos geográficos de referencia (códigos postales, ciudades y estados) para calcular logísticas de envío.
- Tipo de tabla: Maestra (Estática / Precargada).

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
zip_code_prefix	INT	PK	Sí	—	Código postal de 5 dígitos (Identificador).
location	GEOMETRY	—	Sí	—	Coordenada geográfica decimal de latitud.
city	VARCHAR(50)	—	Sí	—	Nombre oficial de la ciudad.
state	VARCHAR(2)	—	Sí	—	Iniciales o siglas de dos letras del estado.

2. Entidad: customers

- Descripción / Propósito: Registra la información de identificación, contacto y ubicación física de los clientes del e-commerce.
- Tipo de tabla: Maestra (Dinámica).

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
identification_number	VARCHAR(50))	PK	Sí	—	Cédula, DNI o RUT del comprador.
geolocation_zip_code_prefix	INT	FK	Sí	—	Relaciona al cliente con geolocations.
name	VARCHAR(50))	—	Sí	—	Primer y segundo nombre del cliente.
last_name	VARCHAR(50))	—	Sí	—	Apellidos completos del cliente.

phone_number	VARCHAR(10)	—	Sí	—	Teléfono de contacto a 10 dígitos.
address	ADDRESS	—	Sí	—	Dirección exacta de entrega domiciliaria.

3. Entidad: sellers

- Descripción / Propósito: Almacena los datos de los vendedores autorizados dentro de la plataforma de ventas.
- Tipo de tabla: Maestra (Dinámica).

Campo	Tipo de Dato	Llav e	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
id	SERIAL	PK	Sí	Autoincrement	Identificador único de vendedor.
zip_code_prefix	INT	FK	Sí	—	Ubicación de despacho vinculada a geolocations.

name	VARCHAR(50)	—	Sí	—	Nombre comercial o de la persona vendedora.
last_name	VARCHAR(50)	—	Sí	—	Apellido del comerciante (si aplica).
phone_number	VARCHAR(10)	—	Sí	—	Teléfono para soporte o despacho.

4. Entidad: categories

- Descripción / Propósito: Clasificación general de los bienes que se comercializan en la tienda virtual.
- Tipo de tabla: Maestra (Controlada).

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
id	SERIAL	PK	Sí	Autoincrement	Identificador numérico de la categoría.

name	VARCHAR(100)	—	Sí	—	Nombre único de categoría (No se repite).
------	--------------	---	----	---	---

5. Entidad: products

- Descripción / Propósito: Contiene las especificaciones técnicas, dimensiones y categorías asociadas a cada artículo en venta.
- Tipo de tabla: Maestra (Dinámica).

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
id	SERIAL	PK	Sí	Autoincrement	Código único de producto.
category_id	INT	FK	Sí	—	Relación directa con categories.
name	VARCHAR(100)	—	Sí	—	Nombre comercial del producto.
specifications	JSONB	—	No	—	Especificaciones del producto

photos	TEXT[]	—	No	—
--------	--------	---	----	---

6. Entidad: order_statuses

- Descripción / Propósito: Catálogo cerrado de los estados posibles por los que pasa una compra (Ej: Creado, Pagado, Enviado).
- Tipo de tabla: Auxiliar / Paramétrica.

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
id	SERIAL	PK	Sí	Autoincrement	Código único de estado.
name	VARCHAR(50)	—	Sí	—	Nombre del estado de orden (Valor único).

7. Entidad: orders

- Descripción / Propósito: Registra la transacción de compra de un usuario y los tiempos clave de la línea logística de entrega.
- Tipo de tabla: Transaccional / Hechos.

Campo	Tipo de Dato	Llave	NoNu lo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
id	SERIAL	PK	Sí	Autoincrement	Número de orden único global.
customer_identification_number	VARCHAR(50)	FK	Sí	—	Identificador del cliente que compró (customers).
order_status_id	INT	FK	Sí	—	Estado del flujo del ciclo de vida (order_statuses).
purchase_timestamp	TIMESTAMP	—	Sí	—	Fecha y hora exacta de creación del carrito.
approved_at	TIMESTAMP	—	No	—	Instante en que la pasarela aprueba el pago.

delivered_carrier_date	TIMESTAMP	—	No	—	Día en que el vendedor entrega el paquete al correo.
delivered_customer_date	TIMESTAMP	—	No	—	Fecha en que el comprador recibe el paquete en mano.
estimated_delivery_date	TIMESTAMP	—	No	—	Fecha límite de entrega prometida por la plataforma.
created_at	TIMESTAMP	-	Si	-	

8. Entidad: order_items

- Descripción / Propósito: Desglosa cada producto e importe financiero individualizado dentro de un pedido. Une clientes, proveedores y productos.
- Tipo de tabla: Transaccional / Detalle.

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
-------	--------------	-------	--------------	-------------------	---------------------------------

id	SERIAL	PK	Sí	Autoincrement	Identificador único de línea de pedido.
product_id	INT	FK	Sí	—	ID del bien comercializado (products).
seller_id	INT	FK	Sí	—	ID de la tienda que provee el bien (sellers).
order_id	INT	FK	Sí	—	Encabezado transaccional al que pertenece (orders).
quantity	INT	—	Sí	—	Unidades adquiridas del producto (Debe ser > 0).
price	FLOAT	—	Sí	—	Precio unitario del artículo al comprar (Debe ser >= 0).
freight_value	FLOAT	—	Sí	—	Costo de envío tasado para este ítem (Debe ser >= 0).
created_at	TIMESTA MP	-	Si	-	

9. Entidad: payment_types

- Descripción / Propósito: Listado parametrizado de medios de transacción aprobados (Ej: Crédito, Débito, Efectivo, Transferencia).
- Tipo de tabla: Auxiliar / Paramétrica.

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
id	SERIAL	PK	Sí	Autoincrement	Identificador único de medio de pago.
name	VARCHAR(50)	—	Sí	—	Nombre del medio de pago (Único).

10. Entidad: order_payments

- Descripción / Propósito: Registra los métodos de abono y el financiamiento en cuotas aplicados a cada transacción comercial efectuada.
- Tipo de tabla: Transaccional / Detalle de Pagos.

Campo	Tipo de Dato	Llave	No Nulo (NN)	Valor por Defecto	Descripción / Reglas de Negocio
-------	--------------	-------	--------------	-------------------	---------------------------------

id	SERIAL	—	No	Autoincrement	Código secuencial técnico interno.
order_id	INT	PK / FK	Sí	—	Parte 1 de PK. Enlace a la orden (orders).
payment_type_id	INT	FK	Sí	—	Enlace directo con payment_types.
sequential	INT	PK	No	—	Parte 2 de PK. Número correlativo de pago si se divide.
installments	INT	—	No	—	Número de cuotas diferidas elegidas (Debe ser > 0).
value	FLOAT	—	No	—	Monto monetario total pagado en este método (> 0).

Script SQL preliminares

```
-- Crear el tipo de datos ADDRESS

CREATE TYPE address_type AS ( line_1 TEXT, line_2 TEXT, neighborhood TEXT,
directions TEXT );
```

-- 1. Tablas Maestras (Independientes)

```
CREATE TABLE geolocations (  
    zip_code_prefix INT PRIMARY KEY,  
    location GEOMETRY(Point,4326) FLOAT NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    state VARCHAR(2) NOT NULL  
);
```

```
CREATE TABLE order_statuses (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE payment_types (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE categories (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL UNIQUE  
);
```

-- 2. Tablas con Dependencias Simples

```
CREATE TABLE customers (  

```

```
    identification_number VARCHAR(50) PRIMARY KEY,  
    geolocation_zip_code_prefix INT NOT NULL,  
    name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    phone_number VARCHAR(10) NOT NULL,  
    address ADDRESS NOT NULL,  
    FOREIGN KEY (geolocation_zip_code_prefix) REFERENCES  
geolocations(zip_code_prefix)  
);
```

```
CREATE TABLE sellers (  
    id SERIAL PRIMARY KEY,  
    zip_code_prefix INT NOT NULL,  
    name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    phone_number VARCHAR(10) NOT NULL,  
    FOREIGN KEY (zip_code_prefix) REFERENCES  
geolocations(zip_code_prefix)  
);
```

```
CREATE TABLE products (  
    id SERIAL PRIMARY KEY,  
    category_id INT NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    specifications JSONB NOT NULL,
```

```

    photos TEXT[],

    FOREIGN KEY (category_id) REFERENCES categories(id)

);

-- 3. Tablas de Procesos (Pedidos y Relacionados)

CREATE TABLE orders (

    id SERIAL PRIMARY KEY,

    customer_identification_number VARCHAR(50) NOT NULL,

    order_status_id INT NOT NULL,

    purchase_timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    approved_at TIMESTAMP,

    delivered_carrier_date TIMESTAMP,

    delivered_customer_date TIMESTAMP,

    estimated_delivery_date TIMESTAMP,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (customer_identification_number) REFERENCES
customers(identification_number),

    FOREIGN KEY (order_status_id) REFERENCES order_statuses(id)

) PARTITION BY RANGE (created_at);

CREATE TABLE order_items (

    id SERIAL PRIMARY KEY,

    product_id INT NOT NULL,

    seller_id INT NOT NULL,

```

```
order_id INT NOT NULL,  
quantity INT NOT NULL CHECK (quantity > 0),  
shipping_limit_date TIMESTAMP NOT NULL,  
price FLOAT NOT NULL CHECK (price >= 0),  
freight_value FLOAT NOT NULL CHECK (freight_value >= 0),  
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (product_id) REFERENCES products(id),  
FOREIGN KEY (seller_id) REFERENCES sellers(id),  
FOREIGN KEY (order_id) REFERENCES orders(id)  
) PARTITION BY RANGE (created_at);  
  
CREATE TABLE order_payments (  
    id SERIAL PRIMARY KEY,  
    order_id INT NOT NULL,  
    payment_type_id INT NOT NULL,  
    sequential INT NOT NULL,  
    installments INT NOT NULL CHECK (installments > 0),  
    value FLOAT NOT NULL CHECK (value > 0),  
    PRIMARY KEY (id),  
    FOREIGN KEY (order_id) REFERENCES orders(id),  
    FOREIGN KEY (payment_type_id) REFERENCES payment_types(id)  
);
```

Referencias

<https://www.mongodb.com/company/blog/building-with-patterns-the-subset-pattern>

<https://www.mongodb.com/company/blog/building-with-patterns-the-extended-reference-pattern>

<https://www.ibm.com/docs/es/iis/11.5.0?topic=overview-referential-integrity-analysis>

<https://medium.com/@luishrsoares/the-change-data-capture-cdc-design-pattern-fa8d3adc964f>